VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

A

ACTIVITY REPORT

BCS714D – BIG DATA ANALYTICS

ON

"LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform"

Submitted in Partial fulfillment of the Requirements for the VII Semester of the Degree of

Bachelor of Engineering in Computer Science & Engineering

By

MOHAMMED ZAIN I - 1HK23CS098
NASRIN KHATOON - 1HK23CS108

Under the Guidance of
Prof. Najmusher
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026 – 2027

HKBK COLLEGE OF ENGINEERING

NO.22/1, OPPOSITE, MANYATA TECH PARK RD, NAGAWARA, BENGALURU, KARNATAKA 560045

HKBK COLLEGE OF ENGINEERING

NO.22/1, OPPOSITE MANYATA TECH PARK RD, NAGAWARA, BENGALURU, KARNATAKA 560045

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the activity work entitled **"LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform"** carried out by Mr. MOHAMMED ZAIN I USN 1HK23CS098 and Ms. NASRIN KHATOON USN 1HK23CS108, bonafide students of HKBK College of Engineering, in partial fulfillment for the award of **Bachelor of Engineering** in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2026-2027. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report. The activity report has been approved as it satisfies the academic requirements in respect of **BCS714D – Big Data Analytics** prescribed for the said Degree.

Prof. Najmusher	**Dr. Smitha Kurian**	**Dr. Mohammed Riyaz Ahmed**
Assistant Professor	Prof. & Head	Principal
Dept. of CSE, HKBKCE	Dept. of CSE, HKBKCE	HKBKCE

DECLARATION

We, the students of 7th semester of Computer Science and Engineering, HKBK College of Engineering, Bengaluru, declare that the activity work entitled **"LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform"** has been successfully completed under the guidance of **Prof. Najmusher**, Department of Computer Science and Engineering, HKBK College of Engineering, Bengaluru. This activity report is submitted in partial fulfillment of the requirements of the course **BCS714D – Big Data Analytics** for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2026 – 2027. Further, the matter embodied in this report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: Bengaluru

Date:

Team members:

Name (USN)	**Signature**
MOHAMMED ZAIN I (1HK23CS098)	_____
NASRIN KHATOON (1HK23CS108)	_____

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to **HKBK College of Engineering, Bengaluru**, for providing us with a platform to pursue our studies and carry out this activity work.

We extend our heartfelt thanks to **Sri. C. M. Ibrahim**, Chairman, and **Sri. C. M. Faiz Mohammed**, Director, HKBKCE, for their vision, leadership, and unwavering commitment to fostering a quality educational environment that has enabled us to grow and excel academically.

It gives us immense pleasure to express our deep sense of gratitude to **Dr. Mohammed Riyaz Ahmed**, Principal, HKBKCE, Bengaluru, for his constant encouragement and support.

We would also like to extend our heartfelt thanks to **Dr. Smitha Kurian**, Prof. & Head of the Department, Computer Science and Engineering, HKBKCE, for being a continuous source of support and motivation throughout the duration of this work.

We consider it a privilege and an honour to express our sincere gratitude to our guide, **Prof. Najmusher**, Assistant Professor, Department of Computer Science and Engineering, for the valuable guidance, timely suggestions, and continuous encouragement provided throughout this activity.

We also extend our sincere thanks to all the faculty members of the **Computer Science and Engineering Department** who directly or indirectly supported and encouraged us during this work.

Finally, we would like to thank our **parents and friends** for their moral support and constant encouragement throughout the completion of this work.

MOHAMMED ZAIN I (1HK23CS098)
NASRIN KHATOON (1HK23CS108)

ABSTRACT

Retail and small-business lending generates a continuous stream of semi-structured records: borrower profiles, loan applications, approval decisions, monthly repayment schedules, collections activity and audit trails. Many small lenders, co-operative societies and microfinance teams still manage this data in spreadsheets or rigid relational schemas, which makes it slow to answer simple portfolio questions such as how much is overdue this week, which product carries the most risk, or how much a borrower can safely be lent.

This activity presents **LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform**, an end-to-end full-stack application that uses MongoDB's document-oriented NoSQL engine as its data platform. A React 18 and Vite client with Recharts visualisations communicates over a JWT-secured REST API built with Node.js and Express, and Mongoose maps six collections (users, customers, loans, repayments, import jobs and audit logs). Each loan document embeds its own amortisation schedule, so a single read returns the loan together with every installment.

The platform covers the complete loan lifecycle: customer onboarding, loan application with reducing-balance EMI and rule-based risk scoring, admin approval that generates the schedule, EMI repayment recorded inside a multi-document ACID transaction, and an automatically maintained collections queue of overdue and upcoming installments. Portfolio KPIs and charts are computed server-side with aggregation pipelines ($group, $bucket, $lookup, $facet, $unwind); bulk customer data is ingested from CSV in batches of 500 using bulkWrite; and a Query Performance Lab compares an indexed IXSCAN against a forced COLLSCAN using explain("executionStats"), with all amounts denominated in Indian Rupees (₹).

In conclusion, LendFlow delivers a responsive, role-based lending workspace with light and dark themes, eligibility and prepayment calculators, CSV export, team management and a complete audit log, demonstrating how document-oriented storage, indexing and in-database aggregation support both day-to-day lending operations and portfolio analytics.

TABLE OF CONTENTS

Title **Page No.**

Certificate.....i

Declaration.....ii

Acknowledgement.....iii

Abstract.....iv

List of Figures.....vi

1. INTRODUCTION.....1

1.1 Background.....1

1.2 Significance.....1

1.3 Objective.....2

2. LITERATURE SURVEY.....3

2.1 Literature Gap Analysis.....4

3. PROBLEM STATEMENT AND OBJECTIVE OF THE PROJECT.....5

3.1 Problem Statement.....5

3.2 Objectives of the Project.....5

4. SYSTEM DESIGN.....6

4.1 Overall Architecture.....6

4.2 System Modules.....6

4.3 Data Flow & Aggregation Pipelines.....7

4.4 Design Considerations & Performance.....8

4.5 Future Enhancement.....9

5. RESULTS AND SCREENSHOTS.....10

5.1 Functional Evaluation.....18

6. CONCLUSION.....19

7. REFERENCES.....20

LIST OF FIGURES

Figure Caption	Page No.
Fig 4.1: Three-tier architecture of LendFlow	6
Fig 5.1: Main Overview Dashboard Interface	10
Fig 5.2: Customer Directory & Record Management	11
Fig 5.3: Customer 360° Profile	11
Fig 5.4: Loan Applications & Approval Workflow	12
Fig 5.5: New Loan Application with Live Eligibility Check	12
Fig 5.6: Loan Details & Embedded Repayment Schedule	13
Fig 5.7: Record EMI Repayment (Transaction)	13
Fig 5.8: Collections Queue (Overdue & Upcoming)	14
Fig 5.9: EMI Calculator & Amortisation Schedule	14
Fig 5.10: Loan Eligibility Checker (FOIR)	15
Fig 5.11: Portfolio Analytics Dashboard	15
Fig 5.12: MongoDB Query Performance & Indexing Laboratory	16
Fig 5.13: Bulk Customer CSV Import & History	16
Fig 5.14: Executive Summary Report	17
Fig 5.15: Activity Audit Log	17
Fig 5.16: Overview Dashboard in Dark Mode	18
Table 4.1: Aggregation pipelines behind the analytics screens	8
Table 5.1: Functional evaluation results	18

1. INTRODUCTION

Lending is one of the most data-intensive activities in finance. Every loan produces a chain of records that keeps growing for its entire tenure: the borrower profile, the application and its risk assessment, the approval decision, an amortisation schedule with one entry per month, every repayment, every follow-up for a missed installment, and an audit trail of who changed what. A modest portfolio of a few thousand loans with three- to five-year tenures therefore generates hundreds of thousands of installment records, and the questions that managers ask of this data, such as the overdue amount by product, the risk mix of new applications or the collection efficiency of the month, require aggregating across all of it.

Traditional tools struggle with this workload. Spreadsheets break down as soon as several officers work on the same data, and they offer no access control, validation or history. Rigid relational schemas split a single loan across several normalised tables (loan header, installment lines, payments) and must re-join them for every screen and every report. Document-oriented NoSQL databases such as MongoDB model a loan the way the business thinks about it, as one document that embeds its own schedule, while still providing compound B-tree indexes, an expressive aggregation framework and multi-document ACID transactions for operations that must never be left half-done.

1.1 Background

In India, formal credit is delivered not only by large banks but also by non-banking financial companies, co-operative societies, microfinance institutions and small fintech lenders. These organisations typically issue standardised products, namely personal, home, vehicle, education and business loans, whose repayment follows the reducing-balance Equated Monthly Installment (EMI) method. For each loan the lender must decide whether the borrower can afford the EMI, track every installment until closure, and follow up when payments are missed.

In big data analytics, this operational environment shows the classic three V’s: Volume (each loan contributes one record per month of tenure), Velocity (repayments and new applications arrive concurrently from several officers every day) and Variety (borrower attributes, product parameters and notes differ between products and change over time). A system for this domain must store evolving documents, find them quickly by status, customer and date, and summarise them on demand without exporting data to a separate analytics tool.

1.2 Significance

The LendFlow platform holds practical significance for lending operations and for data-engineering education. It delivers meaningful contributions across several key areas:

- **Faster, Safer Credit Decisions:** every application is scored for debt-to-income ratio and credit band when it is created, and the application form shows a live eligibility estimate, so officers see immediately when a requested amount exceeds what the borrower can afford.
- **Reliable Repayment Accounting:** recording an EMI updates the loan, inserts a repayment document and closes the loan when the last installment is paid, all inside

one MongoDB transaction.

- **Proactive Collections:** an $unwind-based query lists every overdue and upcoming installment with days past due and contact details, turning raw schedules into a daily work queue.
- **Database Performance Optimization:** a Query Performance Lab compares compound-indexed lookups (IXSCAN) with full collection scans (COLLSCAN) using MongoDB explain plans.
- **Governance and Transparency:** role-based access (admin and officer), bcrypt-hashed passwords, rate limiting and an audit log of every write support accountability.

1.3 Objective

The primary objective of the LendFlow project is to design, implement and evaluate a full-stack loan management and lending analytics platform powered by MongoDB. The specific objectives are as follows:

- To design MongoDB document schemas (customers, loans with embedded installments, repayments, importjobs, auditlogs, users) equipped with compound and unique B-tree indexes.
- To implement reducing-balance EMI, amortisation, debt-to-income risk classification, FOIR-based eligibility and prepayment calculations as a tested business-logic module.
- To record repayments atomically using multi-document transactions on a replica set.
- To compute dashboards and reports with multi-stage aggregation pipelines using $group, $bucket, $lookup, $facet and $unwind.
- To ingest bulk customer data from CSV files in validated batches using bulkWrite, with a persistent import history.
- To establish a Query Performance Lab that benchmarks indexed and unindexed execution via explain("executionStats").
- To deliver a modern, responsive React 18 dashboard with role-based navigation, light and dark themes, Recharts visualisations and Indian Rupee (₹) formatting.

2. LITERATURE SURVEY

This chapter reviews the literature and technical sources that informed the design of LendFlow: data models for operational databases, NoSQL and document stores, MongoDB schema and aggregation design, credit-risk assessment, and the security mechanisms used for authentication.

Codd (1970) introduced the relational model, in which data is represented as normalised relations and recombined at query time through joins. The model provides strong consistency guarantees and a declarative query language, and it remains the foundation of most core-banking systems. Its cost for a loan ledger is that a single business object, a loan with its schedule and payments, is split across several tables and must be re-assembled by joins for every view.

Cattell (2011) surveyed scalable SQL and NoSQL data stores and categorised them into key-value stores, document stores, extensible record stores and scalable relational systems. The survey highlights that document stores such as MongoDB trade fixed schemas and cross-table joins for flexible, nested records, secondary indexes and horizontal scalability, which suits workloads dominated by reads and writes of whole aggregates.

Han, Haihong, Le and Du (2011) presented a survey of NoSQL databases that explains the CAP-theorem trade-offs behind them and compares representative systems. They identify document databases as a good fit for applications whose records are self-contained and semi-structured, and note that the richer query support of document stores narrows the gap with relational systems.

Bradshaw, Brazil and Chodorow (2019), in MongoDB: The Definitive Guide, describe the schema-design principle of embedding data that is read together and referencing data that is shared or unbounded, together with compound index design, the aggregation framework and the multi-document transactions introduced in MongoDB 4.0. LendFlow follows this guidance by embedding the bounded installment schedule (at most 360 entries) inside each loan while referencing customers and storing repayments as separate documents.

The MongoDB documentation on aggregation, indexes, transactions and query plans specifies the operators used in this project: $group, $bucket, $lookup, $facet, $unwind and $dateToString for aggregation; compound and unique B-tree indexes; replica-set transactions; and explain("executionStats"), which reports totalDocsExamined, totalKeysExamined and executionTimeMillis for the winning plan.

Thomas (2000) surveyed credit and behavioural scoring methods for consumer lending, covering statistical, operational-research and machine-learning approaches to forecasting the risk of default, and Siddiqi (2006) describes the practical development of credit-risk scorecards. Both emphasise that affordability, captured by measures such as the debt-to-income ratio, and the borrower’s credit history are dominant inputs to lending decisions. LendFlow adopts a transparent rule-based version of these ideas, which suits an academic platform built on synthetic data.

Provos and Mazières (1999) proposed bcrypt, an adaptive password-hashing scheme whose cost factor can be increased as hardware becomes faster, and Jones, Bradley and Sakimura (2015) standardised the JSON Web Token (RFC 7519) for transferring signed claims. LendFlow stores bcrypt hashes with a cost of 12 and issues signed JWTs that carry the user's identity and role for eight hours.

2.1 Literature Gap Analysis

The sources above cover the individual building blocks well: data models, document-store design, aggregation, credit scoring and authentication. Academic work on lending, however, tends to treat the scoring model in isolation, while database literature demonstrates indexing and aggregation on generic datasets. Few student-level systems combine the two into an end-to-end application in which the lending workflow itself generates the data, transactions protect the repayment ledger, and the same database computes the portfolio analytics and exposes its query plans. LendFlow bridges this gap by unifying loan origination, servicing, collections, analytics, bulk ingestion and index benchmarking on a single MongoDB deployment behind a role-based web interface.

3. PROBLEM STATEMENT AND OBJECTIVE OF THE PROJECT

3.1 Problem Statement

Small and mid-sized lenders need to manage the full life of every loan, from application to closure, while also answering portfolio-level questions quickly. Existing approaches based on spreadsheets or rigid relational schemas suffer from four critical deficiencies:

- **Fragmented Loan Records:** a loan, its schedule and its payments are stored in separate places, so every screen and report must re-assemble them and the data easily drifts out of sync.
- **Unsafe Repayment Posting:** updating the installment, the paid total and the payment ledger in separate steps risks partial updates when an error occurs, leaving the books inconsistent.
- **Analytical Latency:** overdue amounts, risk mix and product performance are computed by exporting data to spreadsheets, so they are always out of date and are rarely computed at all.
- **Weak Governance:** there is no role separation between officers and approvers, no audit trail and no protection of credentials.

3.2 Objectives of the Project

To systematically solve the problems identified above, the LendFlow platform establishes the following concrete technical objectives:

- 1. Design MongoDB collections for users, customers, loans (with embedded installments), repayments, import jobs and audit logs, validated by Mongoose schemas.
- 2. Create compound indexes on loans {status, createdAt}, {customer, createdAt} and {type, status}, and unique indexes on repayments {loan, installment} and users {email}.
- 3. Implement EMI, amortisation, risk, eligibility and prepayment calculations in one module covered by automated unit tests.
- 4. Generate the amortisation schedule on approval and record repayments inside a multi-document transaction that closes the loan automatically when it is fully paid.
- 5. Build a collections queue that lists overdue and upcoming installments for a configurable window of 7 to 90 days.
- 6. Compute dashboard KPIs and charts using $group, $bucket, $lookup, $facet and $unwind pipelines.
- 7. Ingest customer CSV files of up to 5 MB, validating each row and inserting valid rows in batches of 500 with bulkWrite.
- 8. Provide a Query Performance Lab that compares IXSCAN and COLLSCAN execution statistics for the same query.
- 9. Secure the API with bcrypt, JWT, role checks, Helmet headers, CORS and rate limiting, and log every write to an audit collection.

4. SYSTEM DESIGN

The LendFlow platform follows a decoupled three-tier architecture that separates presentation, application logic and data storage. This chapter details the overall architecture, the functional modules, the data flow and aggregation pipelines, the design considerations and the planned future enhancements.

4.1 Overall Architecture

The system comprises three principal operational tiers: the Client Tier (frontend), the Application Server Tier (backend) and the Database Engine Tier (MongoDB).

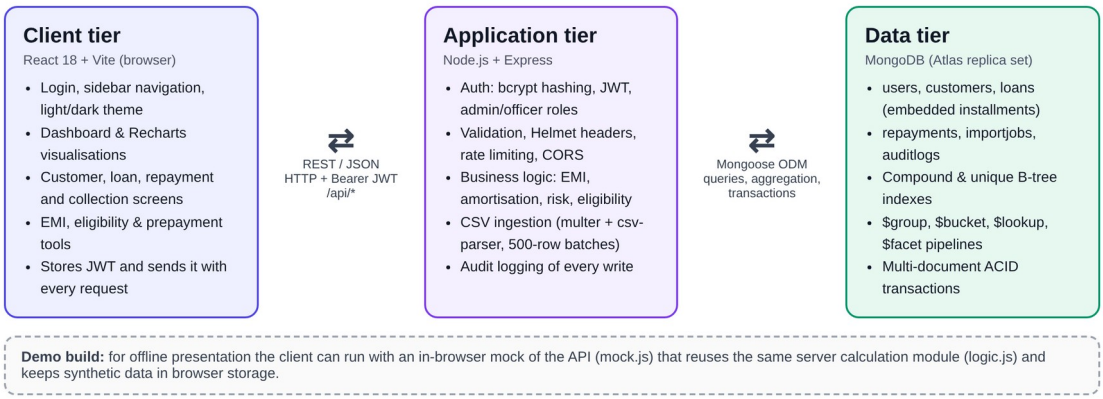


Fig 4.1: Three-tier architecture of LendFlow

- **Client Tier (React 18 + Vite):** a single-page application with grouped sidebar navigation, Recharts visualisations, modal forms and light/dark themes. It communicates exclusively through asynchronous REST requests to /api and sends the session JWT with every request.
- **Application Server Tier (Node.js + Express):** manages API routing, JWT and role verification, request validation, Helmet security headers, CORS and rate limiting (400 requests per 15 minutes), and implements the lending logic. CSV uploads are received with multer and parsed with csv-parser.
- **Database Engine Tier (MongoDB Atlas / Local):** stores the users, customers, loans, repayments, importjobs and auditlogs collections, maintains compound and unique B-tree indexes, executes aggregation pipelines and runs repayment transactions on a replica set.

4.2 System Modules

The LendFlow platform is organised into eight functional subsystems that work together:

1. Authentication & Access Control Module:

Staff sign in with email and password; passwords are stored as bcrypt hashes and a successful login returns a JWT valid for eight hours. Admins approve or reject loans, delete customers, import data, benchmark queries, view the audit log and manage the team; officers handle customers, applications and repayments.

2. Customer Management Module:

Provides a searchable, paginated customer directory with create, update and admin-only delete (blocked when the customer has loans), and a customer 360° profile showing active borrowing, combined EMIs, total repaid and loan history.

3. Loan Origination & Risk Module:

Creates applications for Personal, Home, Vehicle, Education and Business loans. The server computes the EMI with the reducing-balance formula $EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)$, where r is the monthly rate, and classifies risk as High when the credit score is below 580 or the debt-to-income ratio exceeds 60%, Medium when the score is below 700 or the ratio exceeds 40%, and Low otherwise.

4. Approval & Repayment Module:

Only pending applications can be decided. Approval embeds the full amortisation schedule in the loan document. Repayments run inside session.withTransaction(): the installment is marked paid with $elemMatch and $set, totalPaid is incremented with $inc, a repayment document is inserted, and the loan is closed when every installment is paid.

5. Collections Module:

Unwinds the installments of approved loans, keeps pending ones due within a chosen window (7–90 days), joins customer details with $lookup and flags overdue items with the number of days late.

6. Portfolio Analytics & Reporting Module:

Executes server-side aggregation pipelines for KPIs, monthly trend, status mix, product mix, interest-rate bands, top cities, weekday pattern and risk profile, and produces a printable executive summary with overdue exposure.

7. Data Ingestion & Export Module:

Validates uploaded CSV rows (name, email, income, credit score range) and inserts valid rows in batches of 500 with unordered bulkWrite, logging each upload to importjobs. Customers and loans can be exported to CSV with formula-injection neutralisation.

8. Query Performance & Loan Tools Module:

The Query Lab runs the same approved-loans query with an index hint and with a forced $natural scan using explain("executionStats"). Loan Tools provide an EMI calculator with amortisation schedule, an eligibility checker based on a credit-score-dependent FOIR limit (30%, 40%, 50% or 55% of income) and a prepayment planner.

4.3 Data Flow & Aggregation Pipelines

The end-to-end data flow operates through a validated pipeline sequence:

- Step 1: An officer creates customers manually or an admin imports them from CSV; the server validates each record and writes it to MongoDB.

- Step 2: Loan applications are created with computed EMI, risk and DTI; on approval the amortisation schedule is embedded in the loan document.
- Step 3: Repayments are posted in transactions, updating the embedded installment, totalPaid and the repayments collection atomically.
- Step 4: Client dashboard requests trigger targeted aggregation pipelines on the Node.js backend, dispatched in parallel with Promise.all.
- Step 5: MongoDB filters documents through indexed $match stages and computes $group, $bucket, $lookup, $facet and $unwind stages inside the database engine.
- Step 6: Only summarised results are returned; the React client renders KPI cards and Recharts visualisations in Indian Rupees (₹).

Table 4.1: Aggregation pipelines behind the analytics screens

Visualisation	Pipeline stages	Output
Portfolio KPIs	$group with $sum, $avg and $cond	Applications, value, approved value, collected, average EMI
Status / product / risk mix	$group by status, type or risk; $sort	Count and principal per group
Monthly trend	$group by $dateToString("%Y-%m"); $sort; $limit	Applications per month
Interest-rate bands	$facet → $bucket on annualRate	Count and value per band
Top cities	$facet → $lookup, $unwind, $group, $limit 10	Loans per city
Weekday pattern	$facet → $group by $dayOfWeek	Applications per weekday
Collections / overdue	$match, $unwind, $match, $sort, $lookup, $project	Due and overdue installments

4.4 Design Considerations & Performance

To guarantee performance and reliability under growing workloads, several design principles were enacted:

- **Index Design:** compound indexes {status: 1, createdAt: -1}, {customer: 1, createdAt: -1} and {type: 1, status: 1} match the filters actually used, and unique indexes on {loan, installment} and {email} prevent double payment and duplicate accounts.
- **Bounded Embedding:** schedules are capped at 360 installments, keeping loan documents well within MongoDB's 16 MB document limit while allowing a single read per loan.
- **Consistency:** multi-document transactions on a replica set ensure that a repayment is either fully recorded or not recorded at all.
- **Security & Sanitisation:** bcrypt (cost 12), JWT with 8-hour expiry, role middleware, Helmet headers, CORS restricted to the client origin, regex-escaped search, whitelisted update fields and CSV formula-injection neutralisation.
- **Rate Limiting & Validation:** express-rate-limit (400 requests per 15 minutes), 1 MB JSON bodies, 5 MB CSV uploads and Mongoose validators on every write.

4.5 Future Enhancement

- Replace the rule-based risk score with a statistical or machine-learning scorecard trained on historical repayment behaviour.
- Send automated SMS, WhatsApp or email reminders from the collections queue and accept online payments through a payment gateway.
- Add penal interest, late fees, part-payments and loan restructuring.
- Shard the loans and repayments collections for very large portfolios and use change streams for real-time dashboards.

5. RESULTS AND SCREENSHOTS

The LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform was fully built and exercised end to end. All functional modules (Overview Dashboard, Customers, Loans, Repayments, Collections, Loan Tools, Analytics, Reports, Data Import, Query Lab, Team and Audit Log) work as designed. This chapter presents the system's graphical user interfaces, captured on synthetic data of about 300 customers and loans, followed by the functional evaluation.

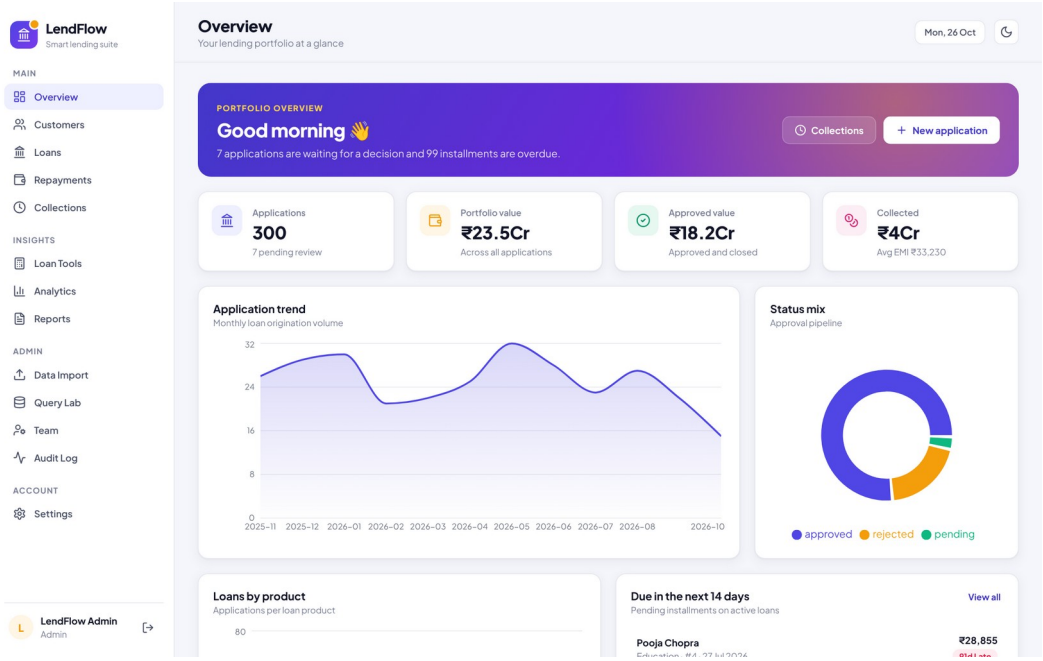


Fig 5.1: Main Overview Dashboard Interface

The Overview Dashboard shows pending decisions and overdue installments, four KPI cards (applications, portfolio value, approved value and amount collected) and charts for the monthly application trend and the status mix, all computed with $group aggregations.

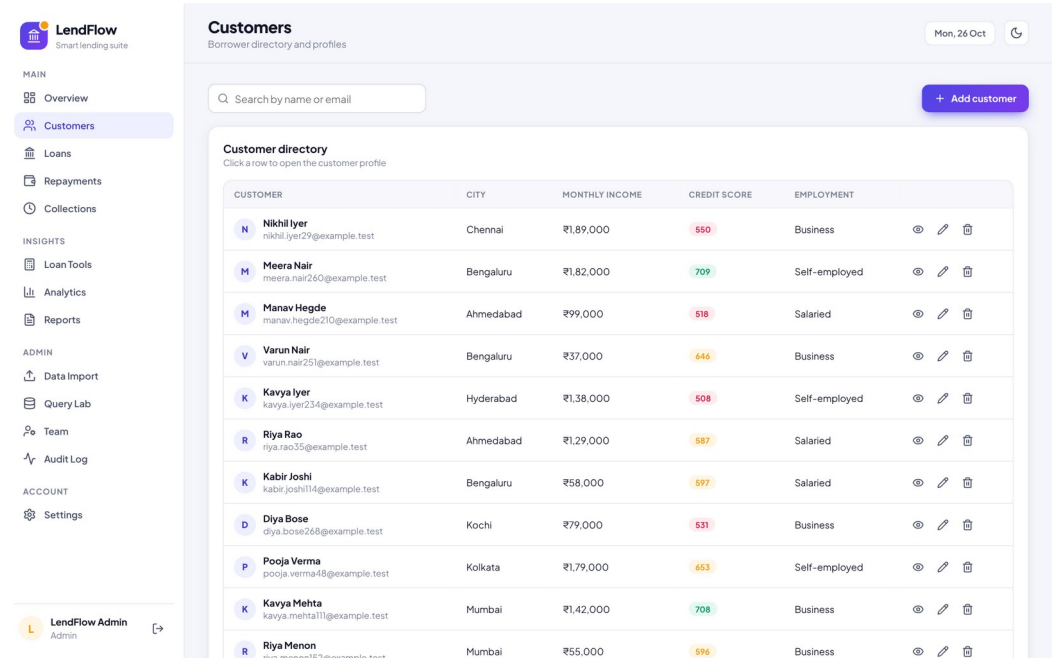


Fig 5.2: Customer Directory & Record Management

A searchable, paginated list of borrowers with city, income, colour-coded credit score and employment; officers view and edit records and admins can delete customers who have no loans.

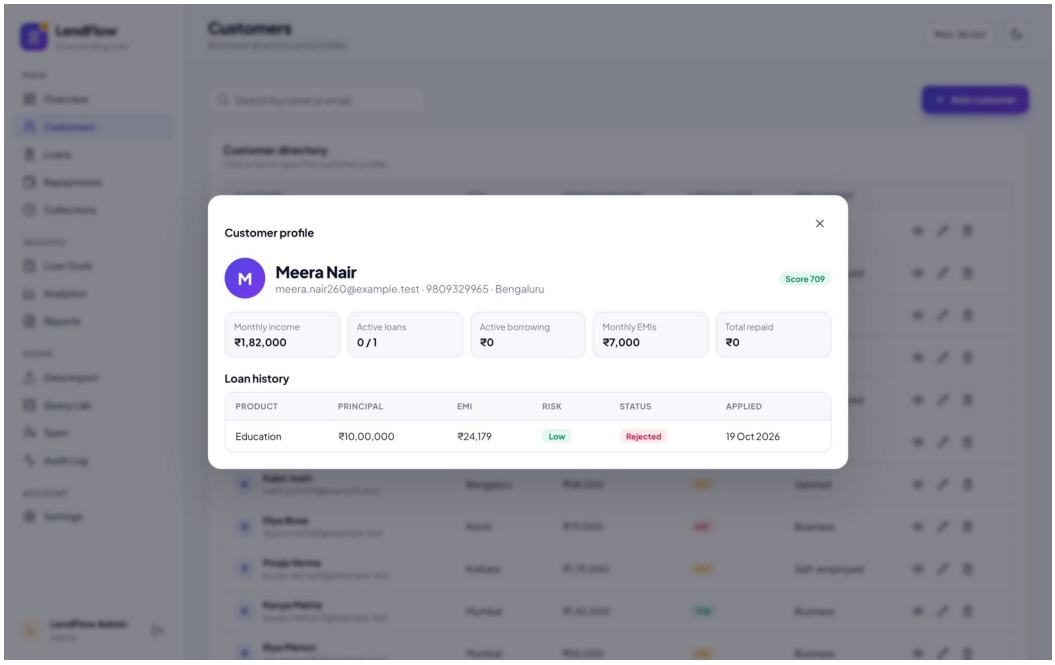


Fig 5.3: Customer 360° Profile

The profile combines the customer document with all of the customer's loans, found with the {customer, createdAt} index, showing active borrowing, combined EMIs, total repaid and loan history.

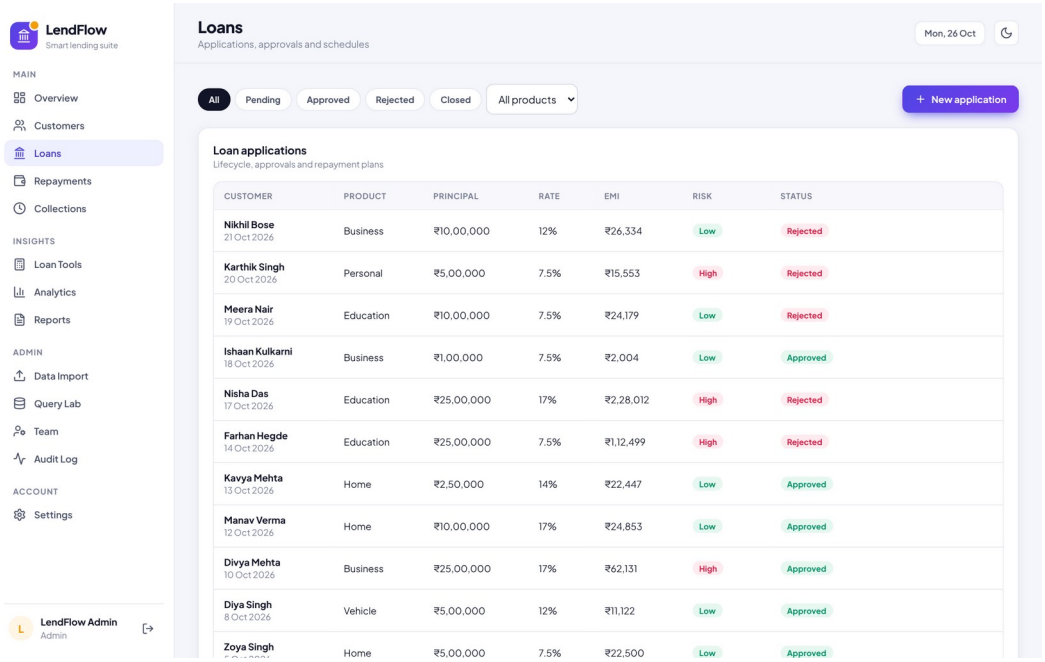


Fig 5.4: Loan Applications & Approval Workflow

Applications are filtered by status and product; each row shows principal, rate, EMI, risk band and status, and admins approve or reject pending applications directly from the list.

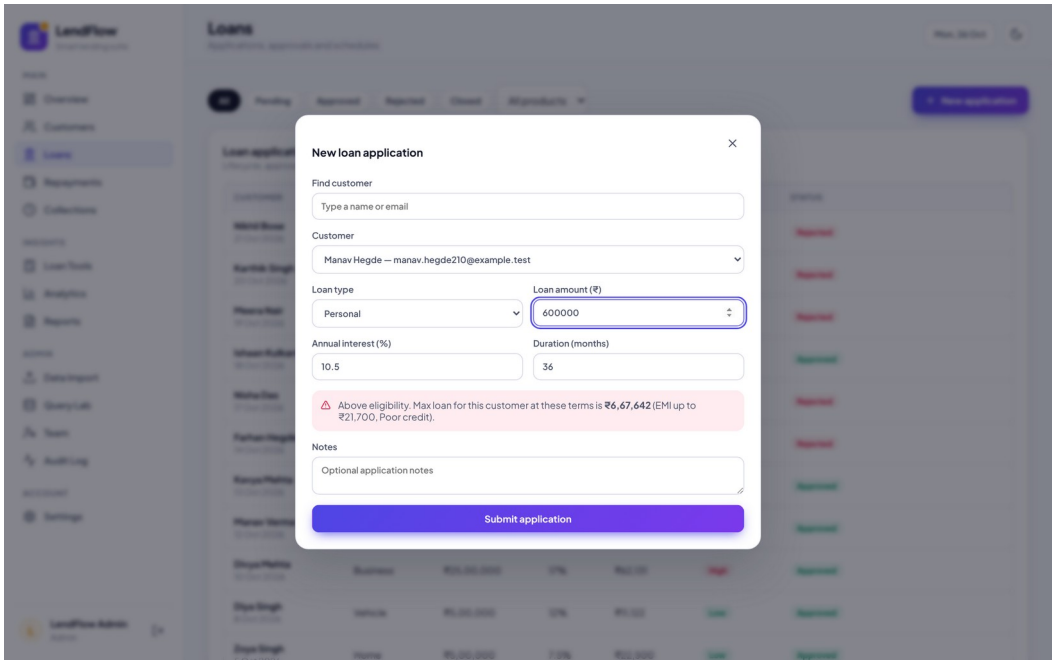


Fig 5.5: New Loan Application with Live Eligibility Check

While the officer enters the amount, rate and tenure, the eligibility endpoint is called for the selected customer and warns when the request exceeds the maximum affordable amount.

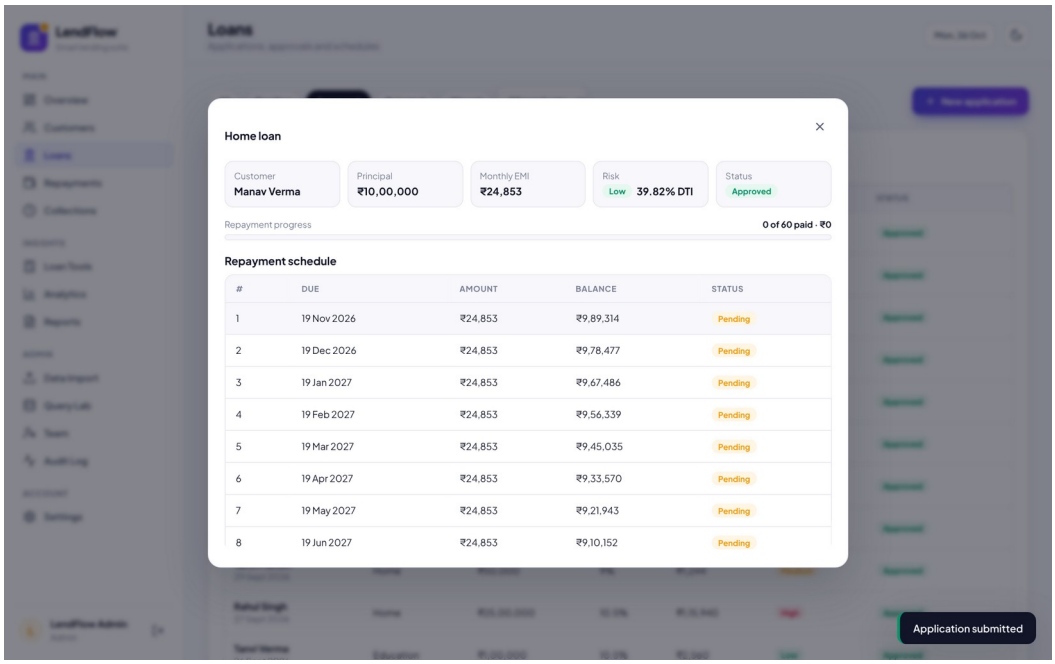


Fig 5.6: Loan Details & Embedded Repayment Schedule

An approved loan shows its embedded amortisation schedule, a repayment progress bar and the status of each installment, with past-due installments highlighted.

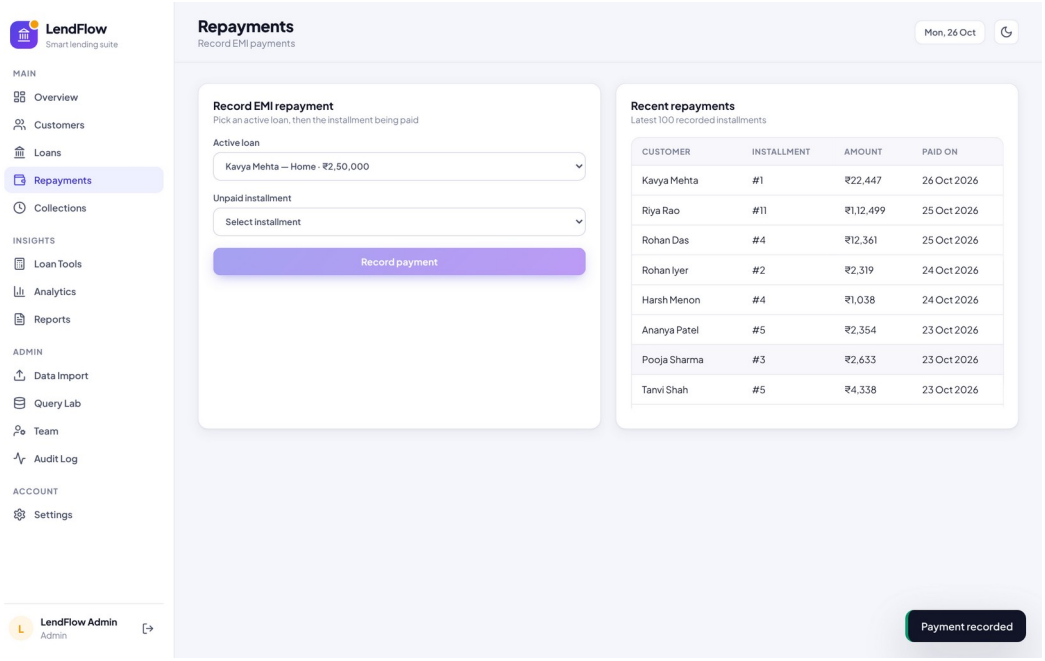


Fig 5.7: Record EMI Repayment (Transaction)

Officers pick an active loan and an unpaid installment; the payment is posted inside a MongoDB transaction and appears in the recent repayments list.

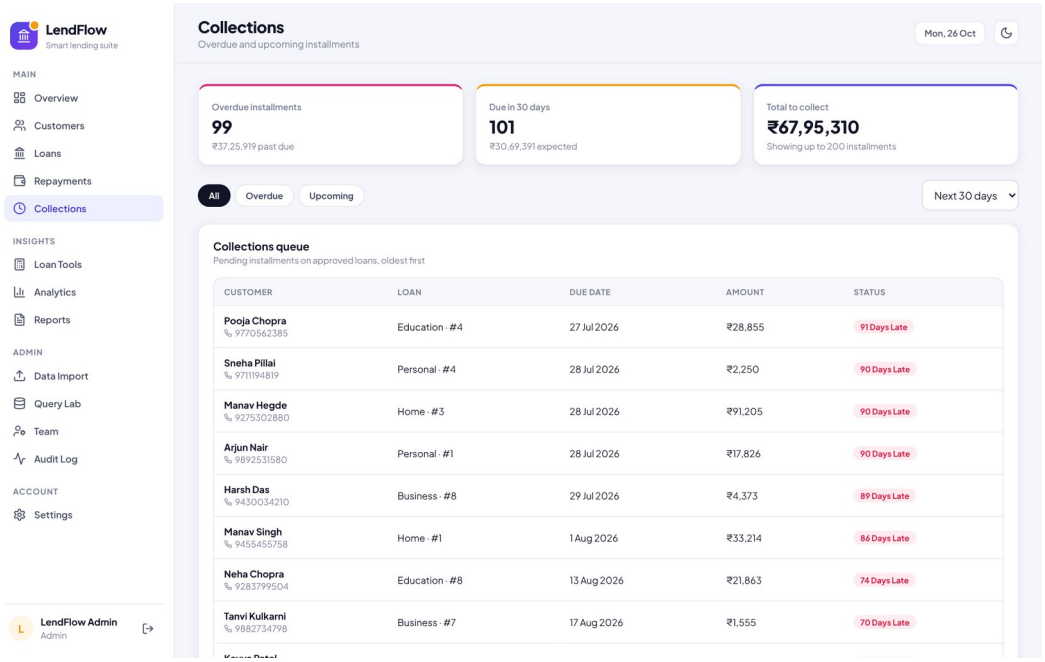


Fig 5.8: Collections Queue (Overdue & Upcoming)

An $unwind-based query lists overdue and upcoming installments within 7–90 days with days past due, contact numbers and totals to collect.

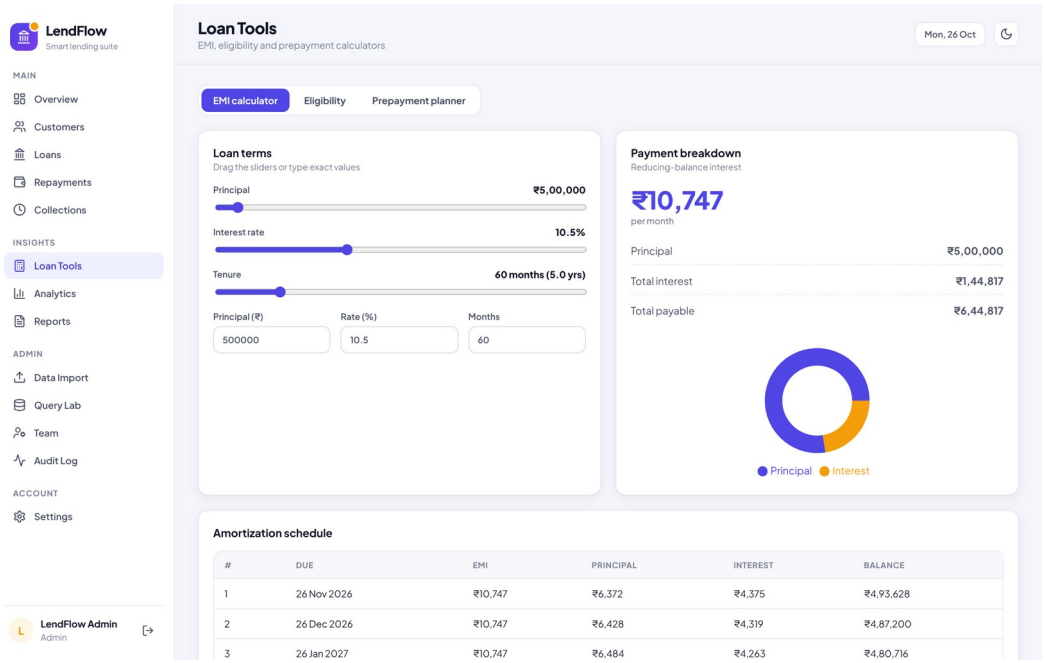


Fig 5.9: EMI Calculator & Amortisation Schedule

Sliders and inputs drive the reducing-balance EMI calculation; the chart compares principal and interest and the full month-by-month schedule is listed.

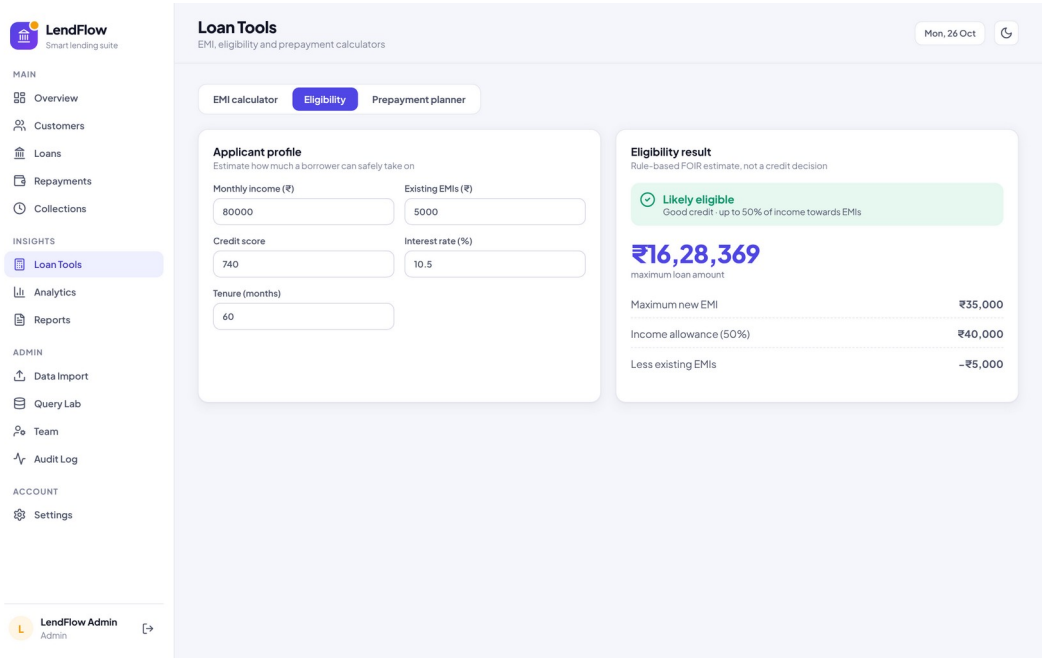


Fig 5.10: Loan Eligibility Checker (FOIR)

From income, existing EMIs, credit score, rate and tenure, the FOIR rule estimates the maximum new EMI and the maximum loan amount.

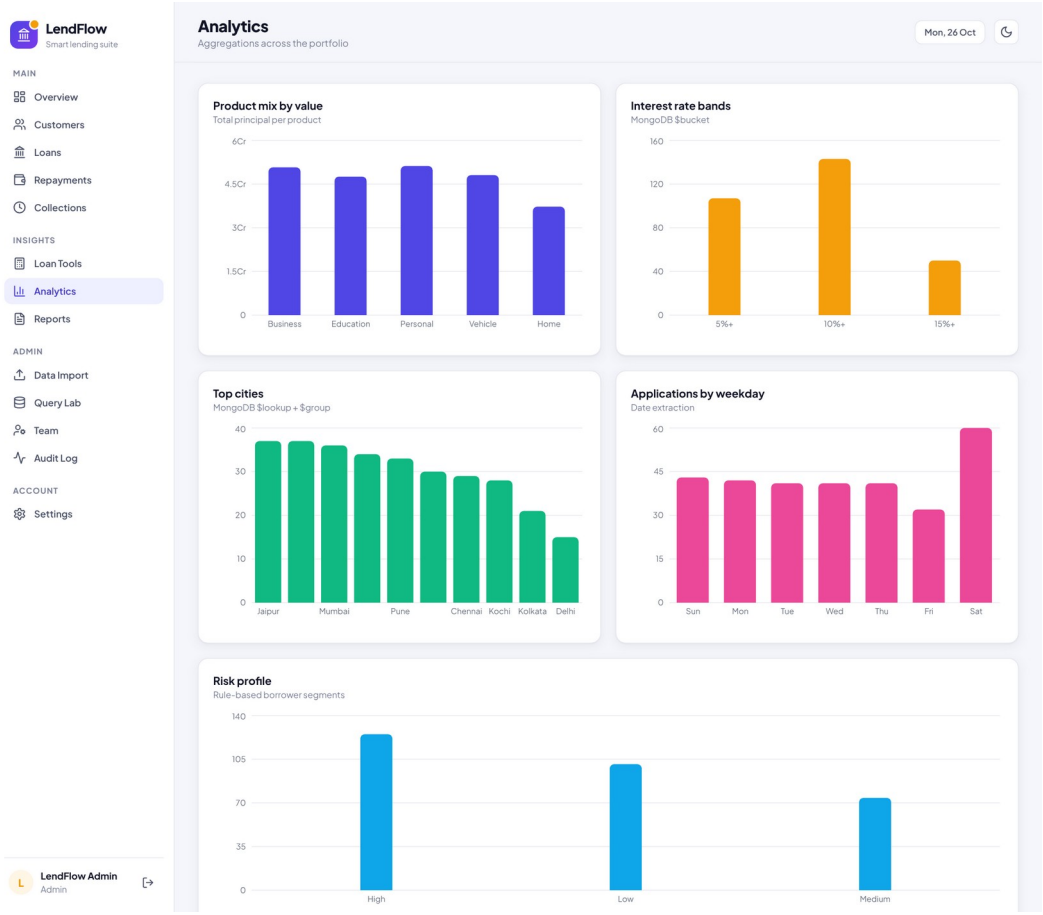


Fig 5.11: Portfolio Analytics Dashboard

Product mix by value ($group), interest-rate bands ($bucket), top cities ($lookup + $group), applications by weekday ($dayOfWeek) and the risk profile of the portfolio.

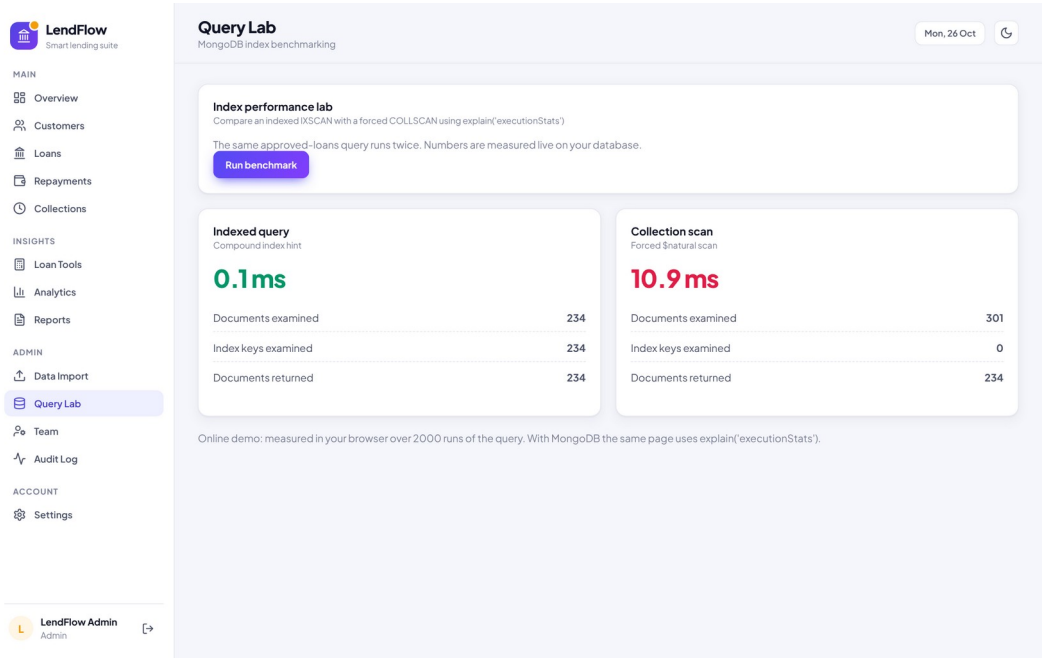


Fig 5.12: MongoDB Query Performance & Indexing Laboratory

The same approved-loans query is run with the compound index and with a forced full scan. With MongoDB the page reports explain("executionStats"); the values in this screenshot come from the in-browser demo build and will differ on a real database.

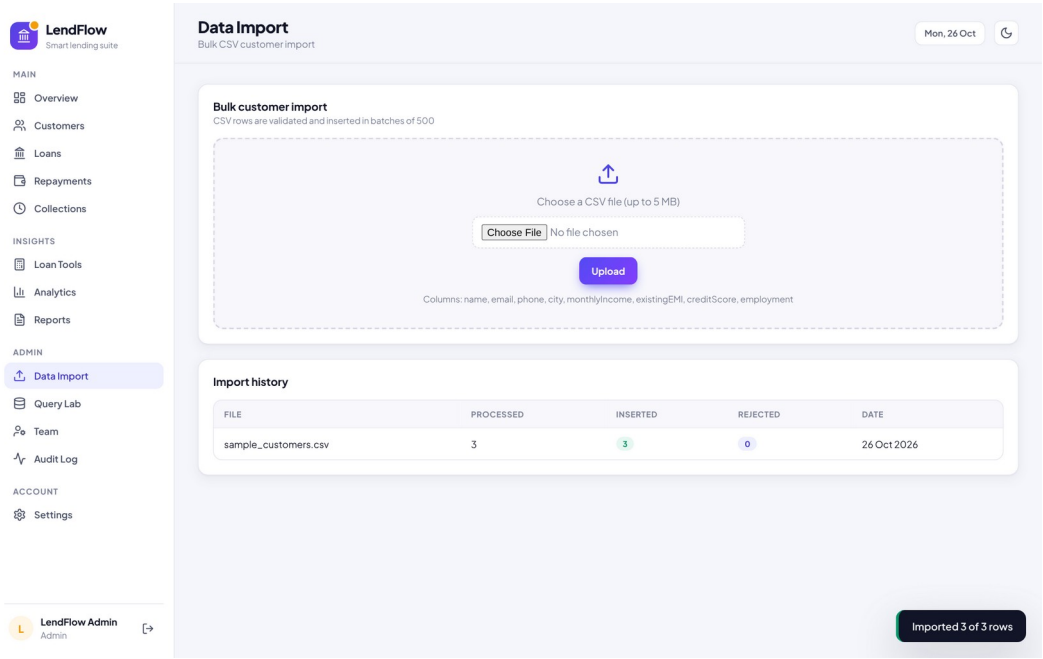


Fig 5.13: Bulk Customer CSV Import & History

Admins upload a CSV file; rows are validated and inserted in batches with bulkWrite, and each upload is recorded with processed, inserted and rejected counts.

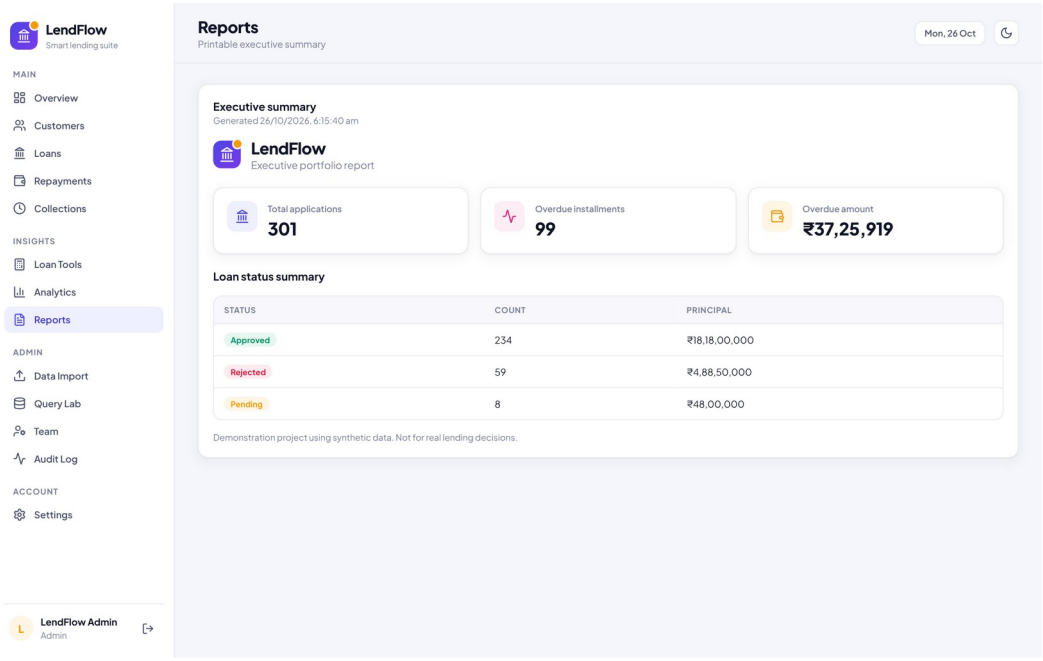


Fig 5.14: Executive Summary Report

A printable executive report with total applications, overdue installments and overdue amount computed with $unwind, plus a loan-status summary.

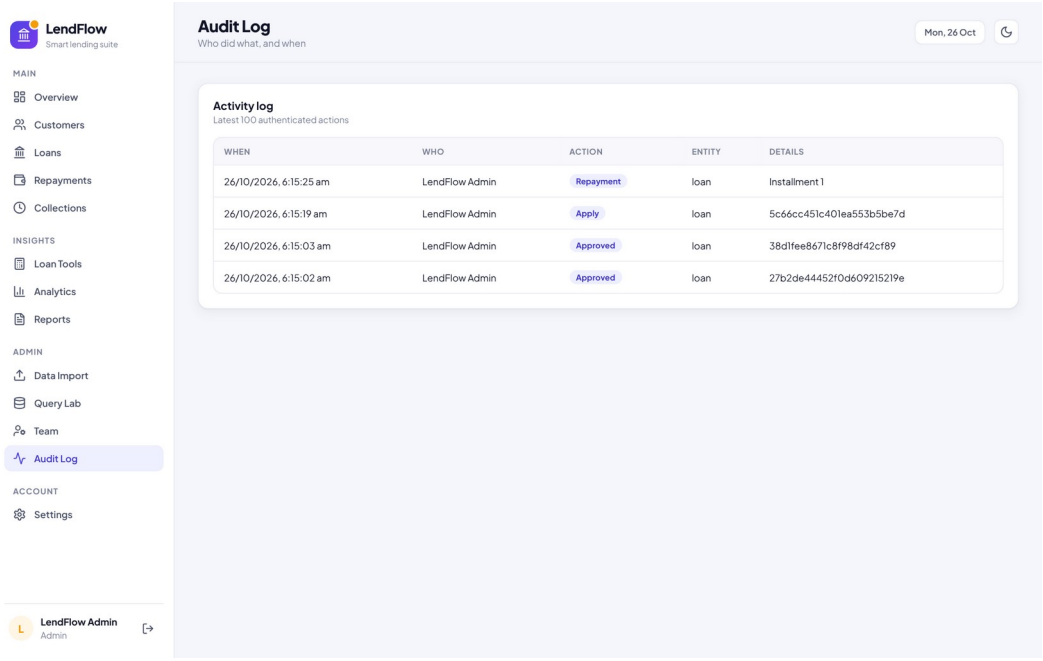


Fig 5.15: Activity Audit Log

Every write operation is recorded with time, user, action, entity and details, giving full traceability of approvals, repayments and data changes.

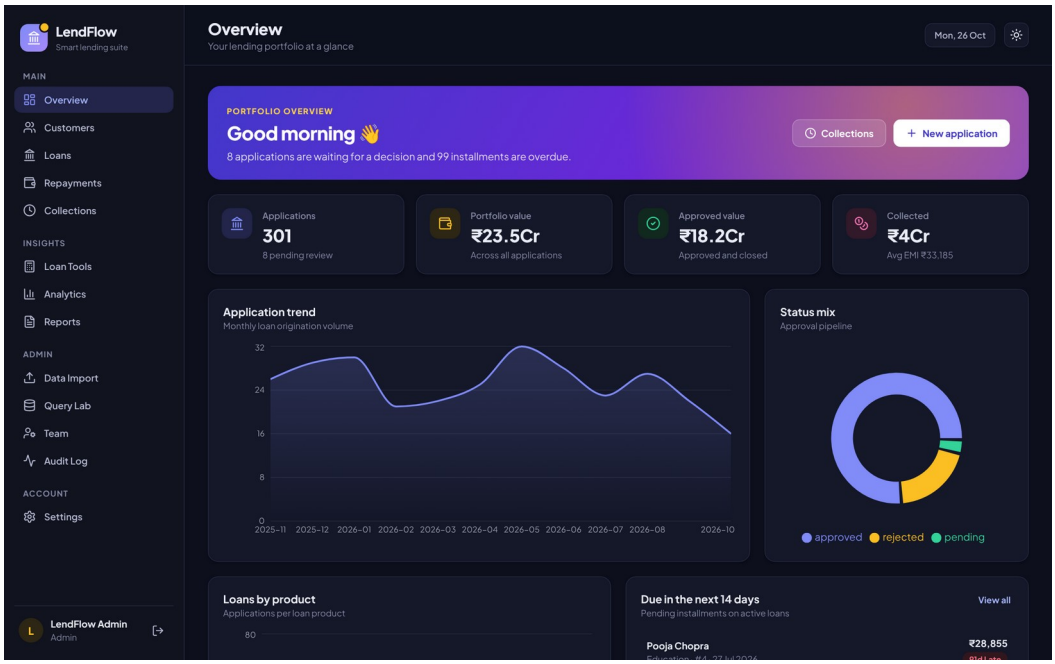


Fig 5.16: Overview Dashboard in Dark Mode

The entire interface supports a dark theme that follows the operating-system preference or the user's choice.

5.1 Functional Evaluation

The business-logic module is covered by automated unit tests run with the Node.js test runner (npm test), and the main user workflows were executed in a Chromium browser against the demo build of the client. Table 5.1 summarises the results.

Table 5.1: Functional evaluation results

Test case	Expected result	Result
EMI at 0% interest (₹12,000 over 12 months)	EMI = ₹1,000	Pass
Amortisation of ₹1,00,000 at 12% for 12 months	Final balance = 0	Pass
High debt-to-income applicant	Risk = High	Pass
Eligibility (income ₹1,00,000, EMIs ₹10,000, score 750)	Max EMI = ₹40,000; EMI of max loan ≈ max EMI	Pass
Credit score below 550	Not eligible	Pass
Prepayment of ₹1,00,000 on ₹5,00,000	Fewer months, interest saved, lower EMI	Pass
Approve a pending application	Status approved; pending count decreases	Pass
Submit application with eligibility hint	Hint shown; application created as pending	Pass
Record an installment payment	Payment recorded and listed	Pass
Add and delete a customer without loans	Customer removed; action audited	Pass
Import sample_customers.csv	3 processed, 3 inserted, 0 rejected	Pass
Reload the page after changes	Session and data persist	Pass

6. CONCLUSION

The LendFlow: MongoDB-Powered Loan Management and Lending Analytics Platform demonstrates that a document-oriented database can serve as the single foundation for both the operational and the analytical sides of a lending business. By modelling each loan as one document with an embedded amortisation schedule, the platform keeps the loan, its installments and its progress consistent and readable in a single operation, while separate repayment and audit collections preserve a complete history.

The implementation shows how MongoDB features map directly onto lending needs. Compound indexes accelerate the status, customer and product filters used throughout the interface; multi-document transactions guarantee that a repayment is either fully recorded or not at all; aggregation pipelines with $group, $bucket, $lookup, $facet and $unwind compute dashboards, collections queues and executive reports inside the database; and bulkWrite ingestion loads customer data in validated batches. The Query Lab makes the effect of indexing visible by comparing documents examined and execution time for IXSCAN and COLLSCAN plans.

Furthermore, the platform gives loan officers and administrators a practical, secure and responsive workspace with live eligibility checks, EMI and prepayment tools, role-based access, CSV import and export, and a full audit trail.

In conclusion, LendFlow fulfils the objectives of the BCS714D Big Data Analytics activity and provides a solid base for future work such as statistical credit scoring, automated reminders, payment-gateway integration and sharded deployment for larger portfolios.

7. REFERENCES

1) E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

2) R. Cattell, “Scalable SQL and NoSQL Data Stores,” ACM SIGMOD Record, vol. 39, no. 4, pp. 12–27, 2011.

3) J. Han, E. Haihong, G. Le and J. Du, “Survey on NoSQL Database,” in Proc. 6th International Conference on Pervasive Computing and Applications (ICPCA), pp. 363–366, 2011.

4) S. Bradshaw, E. Brazil and K. Chodorow, MongoDB: The Definitive Guide, 3rd ed., O’Reilly Media, 2019.

5) MongoDB Inc., “Aggregation Operations,” MongoDB Manual. Available: https://www.mongodb.com/docs/manual/aggregation/

6) MongoDB Inc., “Indexes” and “Transactions,” MongoDB Manual. Available: https://www.mongodb.com/docs/manual/indexes/

7) L. C. Thomas, “A Survey of Credit and Behavioural Scoring: Forecasting Financial Risk of Lending to Consumers,” International Journal of Forecasting, vol. 16, no. 2, pp. 149–172, 2000.

8) N. Siddiqi, Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring, John Wiley & Sons, 2006.

9) N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, FREENIX Track, 1999.

10) M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.

11) React Core Team, “React: The Library for Web and Native User Interfaces.” Available: https://react.dev/

12) Express.js, “Fast, Unopinionated, Minimalist Web Framework for Node.js.” Available: https://expressjs.com/

13) Mongoose, “Mongoose ODM Documentation.” Available: https://mongoosejs.com/docs/

14) Aadhinath5527, “LoanSphere – MongoDB Loan Management & Big Data Analytics” (base academic code that LendFlow extends), GitHub. Available: https://github.com/Aadhinath5527/LoanSphere_FullStack_Project